Graph Execution Overview

GEX = Graph Compiler + Runtime

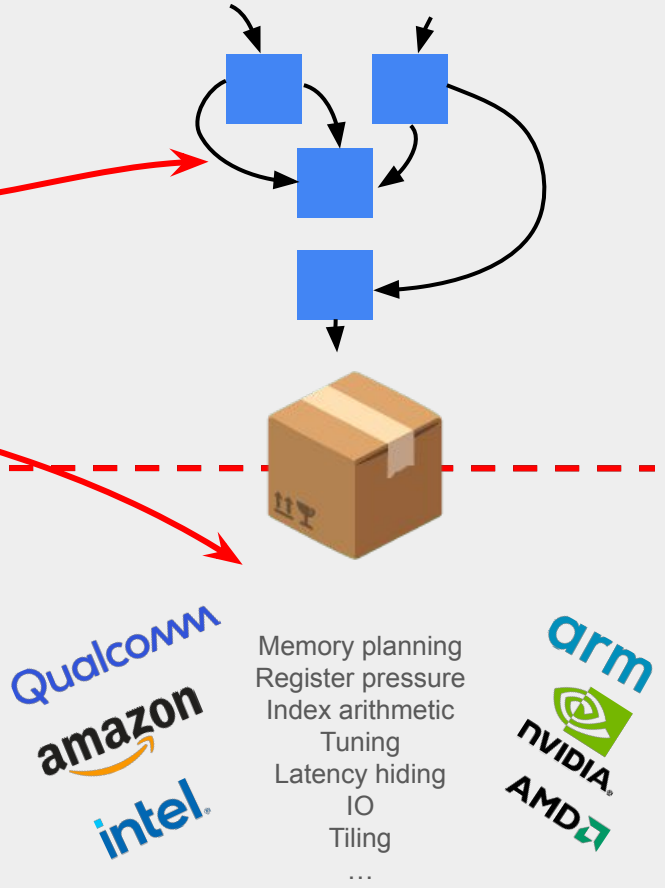
The Graph Compiler

The Rationale

Compiler design, optimizing compilers and metaprogramming

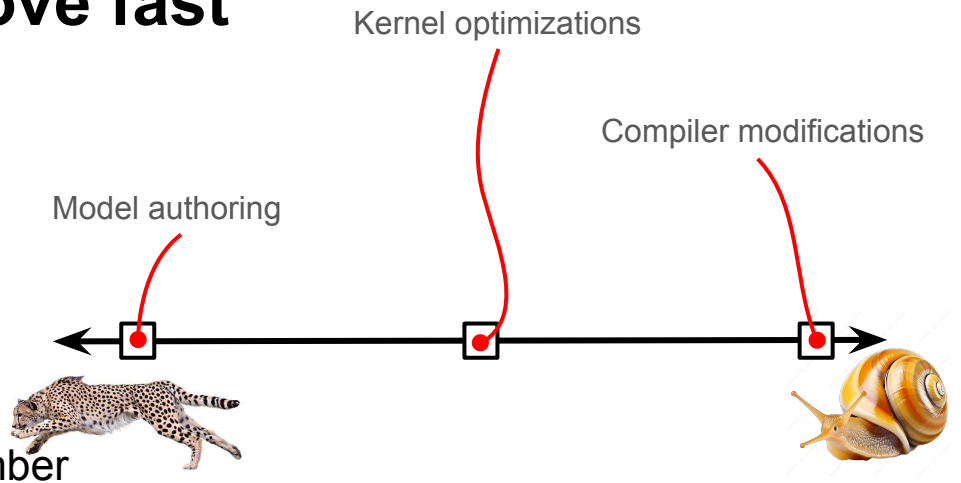
MAX and `mojo`

- Models have semantic understanding but they are not concerned with **low-level** HW details.
- Kernels are unaware of the graph.
- There is both a conceptual **and** language boundary (Python → Mojo).
- Something has to bridge that gap.
- The GC can take liberties in supporting kernel authors by reflecting on both the code and the model, thus acting as a **metaprogramming** extension to the Mojo compiler.
- We control the stack so the compilers can remain simple and greedy.



Modular ~~wants~~ has to move fast

- Modular *was* a startup. Modular *still* needs to move fast.
- GC does not exist in a vacuum (see previous slide).
- Every month:
  - Several new models
  - Flash attention flavour
  - A new optimization trick (remember DeepSeek?)
  - A new chip announced
  - A new potential customer



Compilers are the slowest moving part. You can adjust models and optimize kernels much faster than a compiler can adapt.

No compiler magic

The death of optimizing compilers Daniel J. Bernstein University of Illinois at Chicago & Technische Universiteit Eindhoven

- Mojo and the GC were designed with some of those ideas in mind.
- The GC is **an extension of Mojo**, serving kernels and model authors.
- This could change but we generally avoid:
  - ❌ Large search spaces
  - ❌ Complex or unpredictable optimization decisions
  - ❌ LLM-based compilers
  - ❌ Restrictive programming models (DSLs)
  - ❌ Blocking ourselves from reaching SoL performance
  - ❌ Forcing users to use the GC to get anything done

✅ Small or no search spaces
✅ Predictable and easy to follow optimizations

✅ Traditional algorithms
✅ Mojo (a complete GPPL)

✅ No forced programming model, no limitations

✅ All kernels compile and run without the GC or the rest of MAX

Extensibility as a first class citizen

- We refer to extensibility as our escape hatch.
- It is more of an **additional tool** to achieve your desired goal (performance or functionality).
- The GC is purposefully simple and greedy. Custom ops naturally fill the gaps.

Custom ops are a double edged sword: You want them as an option, not as the default.

```
def MOComposite_FusedConcatSliceOp :
MO_CompositeOp<"concat_slice", [
   %concat = mo.concat[axis](inputs)
   %slice = mo.slice(%concat)

def MOComposite_ReduceRMSNormRoPEOp :
MO_CompositeOp<"rms_norm_rope", [
   normed = rms_norm(input, weight, epsilon,
weight_offset)
   x1, x2 = split(normed, axis=-1)
   rotated = concat(-x2, x1, axis=-1)
   result = normed * cos_vals + rotated * sin_vals

def MOComposite_FusedMatmulAddOp :
MO_DotBase<"composite.matmul_add",
[MO_CompositeOpTrait]> {
   Computes C = A @ B (optionally transposed) +
residual
   and the residual addition into a single kernel call.
```

This one is especially bad

💡 **Takeaway:** On every technical slide, you will find this box. Read it if things aren't clear to you or use as a summary.

The Graph Compiler

The three compilation phases + codegen

💡 **Takeaway:** Simple, modular frontend that models modern AI workloads. Features compose and users can integrate their custom ops easily.

Frontend: mo

A generic ML opset

- All mlir-based
- `rmo` for relaxing shapes in the Python framework
- Core opset is 139 ops
- Symbolic folding and optimizations at this level

```
mo.graph @chain(
 %x: !mo.tensor<[8, 8], f32>,
 %y: !mo.tensor<[8, 8], f32>,
 %z: !mo.tensor<[8, 8], f32>
) -> !mo.tensor<[8, 8], f32> {
 %mm = mo.matmul(%x, %y) :
(!mo.tensor<[8, 8], f32>,
!mo.tensor<[8, 8], f32>) ->
!mo.tensor<[8, 8], f32>
 %sum = mo.add(%mm, %z) :
!mo.tensor<[8, 8], f32>
 %out = mo.relu(%sum) : !mo.tensor<[8,
8], f32>
 mo.output %out : !mo.tensor<[8, 8],
f32>
}
```

Composable features

- Mutable inputs for in-place operations (like KV caches)
- Dynamic dimensions as named attributes
- Shape inference generates algebraic expressions.
- Shape function fallbacks

```
!kvc_ty    = !mo.buffer<[B, M, K, D], f32>
!new_kv_ty = !mo.tensor<[B, S, K, D], f32>
%cur, %ch1 = mo.mutable.load(%kvc, %ch0)
 : (!kv_cache_ty, !mo.chain) -> (!new_kv_ty,
!mo.chain)
%upd = mo.add(%cur, %new_kv) : !new_kv_ty
%ch2 = mo.mutable.store(%kvc, %upd, %ch1) :
(!kvc_ty, !new_kv_ty, !mo.chain) -> !mo.chain
%shape = mo.shape_of(%new_kv)
 : (!new_kv_ty) -> !mo.tensor<[4], si64>
%merged = rmo.reshape(%cur) : (!new_kv_ty) ->
!mo.tensor<[mul_no_wrap(B, M, K, D)], f32>
%flat = mo.add(%merged, %merged)
 : !mo.tensor<[mul_no_wrap(B, M, K, D)], f32>,
   !mo.tensor<[mul_no_wrap(B, M, K, D)], f32>
 ->!mo.tensor<[mul_no_wrap(B, M, K, D)], f32>
```

Escape hatches

- `mo.composite.*` for edge cases that the middle end can't support
- `mo.custom` for arbitrary custom ops
- Fewer = better

```
%0 = mo.composite.matmul_add(%a, %b,
%c) {transpose_b = true} :
 (!mo.tensor<[B, K], f32>,
  !mo.tensor<[M, K], f32>,
  !mo.tensor<[M], f32>) ->
!mo.tensor<[B, M], f32>
%1 = mo.custom {symbol = "my_kernel"}
(%0) : (!mo.tensor<[B, M], f32>) ->
!mo.tensor<[B, M], f32>
```

💡 **Takeaway:** Middle end optimizes (ex: operator fusion) and progressively lowers things into a form that makes it easy to generate Mojo code.

Middle end: map+mogg

map: A fusion dedicated dialect

- Views are index functions
- Elementwise and reductions are generator-based
- Opaque kernels can opt into prologue and epilogue fusion
- Vectorization and shape flattening

```
%add = map.iter.foreach<[mul_no_wrap(M, K)]>
   in  (%x   : !mo.tensor<[M, K], f32>, %y : !mo.tensor<[M, K], f32>)
   out (%out : !mo.tensor<[M, K], f32>)
   idx (%i   : !map.index<1>) -> !mo.tensor<[M, K], f32> {
 %iv = map.index.reshape<#reshape_spec>(%i)
    : !map.index<1> -> !map.index<2>
 %xv = map.load %x[%iv] : !mo.tensor<[M, K], f32>
 %yv = map.load %y[%iv] : !mo.tensor<[M, K], f32>
 %s = map.call.elementwise @linalg.Add(%i, %xv, %yv) : !map.index<1>, f32, f32 -> f32
 map.store %out[%i] = %s : !mo.tensor<[M, K], f32>
}
```

mogg: Progressive lowering

- Fusion IR lands here
- Dumping ground for codegen boilerplate (ex: kernels that consume variadics, scalars, device contexts, etc)
- Just enough IR information to do mojo codegen

```
mogg.call.foreach
@builtin_primitives.primitives._foreach
shape(%dz, %dk) context(%ctx) capturing(%it0, %it1, %ot : !mogg.tensor_, !mogg.tensor_, !mogg.tensor_) attributes {target = "cpu"} {
 ^bb0(%in0 : !mogg.tensor_, %in1 : !mogg.tensor_, %out : !mogg.tensor_, %idx : !map.index<2>):
   %v0 = mogg.tensor.load %in0[%idx] : !mogg.tensor_, !map.index<2> -> f32
   ...
   %sum = mogg.call.elementwise["builtin_kernels.elementwise.Add"] (%v0, %v1) {moOpName = "mo.add"} : f32, f32 -> f32
   ...
 }
```

Mental model

- The middle end has no semantic information: **everything is a custom op**
- Users opt into prologue and epilogue fusion and write new elementwise or reduction ops

```
%epi = map.iter.epilogue (...) {
   %relu = map.call.elementwise @linalg.Relu(%ei, %v) : ...
   map.store %cap[%ei] = %relu : ...
 }

 map.call.opaque @builtin_kernels.linalg.Matmul[transpose_b = false]
   inputs (%A, %B) outputs (%epi)
```

💡 **Takeaway:** Backend is a wrapper for an async runtime. Does optimize but mainly used for device-based orchestration and resource setup like memory/kernels.

Backend: MGP

CPU dataflow runtime

- All operations run on the host (CPU)
- Ops opaquely launch device kernels
- Dynamic dimensions and expressions are converted to SSA values+ops

Coarse grain optimizations

- Optimizations are very generic and uninformed about the kernels
- Static/dynamic memory planning
- EICM
- Device graph synthesis

Culminates into Mojo codegen

- **mojo** is the “target” backend
- Matches the level of abstraction of the GC
- Forms a modular™ compilation chain

```
mgp.model @add (... %x, %y) -> (... %out)
 devices [#M.device_ref<"cpu", 0>]
 init (%wt, %cpu : !mgp.device_context<"cpu", 0>) -> (...) { ... }
execute (..., %cpu, %x, %y) -> (... %out) {
 %xspec = mgp.tensor.extract.tensor_spec %x : <?x?xf32>
 %b = mgp.tensor_spec.get_dim[0] %xspec
 %k = mgp.tensor_spec.get_dim[1] %xspec
 %bs = pop.cast_from_builtin (index.casts %b : index to si64) : si64 to !kgen.scalar<si64>
 %ks = pop.cast_from_builtin (index.casts %k : index to si64) : si64 to !kgen.scalar<si64>
 %bytes = pop.cast_to_builtin (pop.mul %bs, %ks) : !kgen.scalar<si64> to si64  ... index arithmetic ...
 %out_buf = mgp.buffer.alloc %ch0, %bytes, %cpu : !mgp.buffer, !mgp.device_context<"cpu", 0>
 %ch = mogg.generic.compiled.region (%ch0, %x_buf, %y_buf, %out_buf, %bs, %ks, %cpu)
     : (..., %cpu : !mgp.device_context<"cpu", 0>) -> !mef.chain {
   ...
   mogg.call.stub @add_map ... : ..., !mogg.context
   ...
 } fusionInfo = "mo.add"
 mef.output %ch, %out
}
mogg.kernel.stub @add_map (... : !mogg.dim_, ..., !mogg.context) ... { ... }
```

`init` phase

`execute` phase

💡 **Takeaway:** Thanks to Mojo codegen and the full control over MAX,, the GC is equipped with many debugging and quality of life features.

Debugging features

Thanks to Mojo codegen, debugging features are easy to implement:

- **`-mo-nan-check-pass`**: Insert NaN/Inf checks after each compiled region's floating-point outputs.
- **`-mogg-wrap-kernels-with-trace-op`**: Wrap kernels in a with Trace[...] clause
- **`-mogg-add-profile-scope-tracing`**: Synthesizes profiler trace ranges from contiguous that share the same profile scope
- **`-mogg-add-try-except-blocks`**: Gives kernels the ability to raise
- **`-mgp-attach-python-stack-traces`**: Gives kernels the ability to refer back to the **MAX Python line of code that created it** (the full stack trace).

```
def gelu_map(...) raises:
   try:
       ...
       with max.runtime.tracing.Trace[max.runtime.tracing.TraceLevel.OP, target = "cpu"]("gelu", task_id = ...(...)):
           ...
           def lambda_____6[width : Int, element_alignment : Int = 1](idx: layout.Coord) {var}:
               ...
               var var_____2 = builtin_primitives.primitives.tile_tensor_strided_load[...](...)
               var var_____3 = builtin_kernels.elementwise.Gelu.elementwise[dtype = DType.float32](var_____2)
               builtin_primitives.primitives.tile_tensor_strided_store[...](...)
           builtin_primitives.primitives._foreach[...](...)
   except e__5:
       var var___0 = builtin_primitives.primitives.mogg_format_kernel_error("gelu_map", e__5, "  mo.gelu
(/private/tmp/claude-501/-Users-ferasboulala-modular/b14aca36-6985-40b1-a4c3-bea0ff53a2a2/scratchpad/ffn.mlir:25) : (!mo.tensor<[dyn, dyn], f32>) ->
!mo.tensor<[dyn, dyn], f32>\n", "")
       raise var___0^
```

Middle end

Full optimization pipeline explained

💡 **Takeaway:** Simple, readable intermediate representation (IR) of a small synthetic graph.

Input IR

$$z = x + y^T$$

```
mo.graph @add<N>(%x : !mo.tensor<[N, 64], f32>, %y : !mo.tensor<[64, N], f32>) -> !mo.tensor<[N, 64], f32> {
 %p = mo.constant {value = #M.dense_array<1, 0> : tensor<2xsi64>} : !mo.tensor<[2], si64>
 %yt = mo.transpose(%y, %p) : (!mo.tensor<[64, N], f32>, !mo.tensor<[2], si64>) -> !mo.tensor<[N, 64], f32>
 %z = mo.add(%x, %yt) : !mo.tensor<[N, 64], f32>
 mo.output %z : !mo.tensor<[N, 64], f32>
}
```

-mogg-import-packages

```
@extensibility.register("mo.add")
struct Add(ElementwiseBinaryOp):
    """Registers the `mo.add` graph op with the graph
compiler."""

    @staticmethod
    def elementwise[
        dtype: DType,
        width: SIMDLength,
    ](lhs: SIMD[dtype, width], rhs: SIMD[dtype, width]) ->
SIMD[dtype, width]:
        return lhs + rhs
```

```
mogg.kernel.decl @builtin_kernels.elementwise.Add
      registration = "mo.add"
      device_info = <"", "", "", "">
      elementwise = <<!lit.generator<...>>
      struct_params = []
      is_user_defined = false

mo.graph @add<N>(%x : ..., %y : ...) -> ... {
 %p = mo.constant {...} : ...
 %yt = mo.transpose(%y, %p) : ... -> ...
 %z = mo.add(%x, %yt) : ...
 mo.output %z : ...
}
```

💡 **Takeaway:** The GC introspects parsed Mojo (not an abstract syntax tree but the IR itself) to enable certain optimization features like fusion.

💡 **Takeaway:** High level operators are lowered to their kernel representation which is more amenable to generic optimization by the middle end.

-mo-to-map

$$z = x + y^T$$

```
...
   %1 = map.iter.foreach<[N, 64]>
   inputs  (%arg1: !mo.tensor<[64, N], f32>, %0: !mo.tensor<[2], si64>)
   outputs (%arg4: !mo.tensor<[N, 64], f32>)
   index   (%arg5: !map.index<2>)  -> !mo.tensor<[N, 64], f32> {
     %3 = map.index.transpose<#map.index.transpose.spec<[64, N] -> [N, 64], perm = [1, 0]>> (%arg5, %0) : (!map.index<2>,
!mo.tensor<[2], si64>) -> !map.index<2>
     %4 = map.load %arg1[%3] : !mo.tensor<[64, N], f32>
     map.store %arg4[%arg5] = %4 : !mo.tensor<[N, 64], f32>
   } {...}

   %2 = map.iter.foreach<[N, 64]>
   inputs  (%arg0: !mo.tensor<[N, 64], f32>, %1: !mo.tensor<[N, 64], f32>)
   outputs (%arg4: !mo.tensor<[N, 64], f32>)
   index   (%arg5: !map.index<2>)  -> !mo.tensor<[N, 64], f32> {
     %3 = map.load %arg0[%arg5] : !mo.tensor<[N, 64], f32>
     %4 = map.load %1[%arg5] : !mo.tensor<[N, 64], f32>
     %5 = map.call.elementwise @builtin_kernels.elementwise.Add(%arg5, %3, %4) : !map.index<2>, f32, f32 -> f32
     map.store %arg4[%arg5] = %5 : !mo.tensor<[N, 64], f32>
   } {...}
...
```

💡 **Takeaway:** Kernels repeatedly fus until convergence. This fusion pass invokes several fusers, each finding a “kind” of fusion and running independently.

-map-fuse-kernels

$$z = x + y^T$$

```
...
 %1 = map.iter.foreach<[N, 64]>
   inputs  (%arg0: !mo.tensor<[N, 64], f32>, %arg1: !mo.tensor<[64, N], f32>, %0: !mo.tensor<[2], si64>)
   outputs (%arg5: !mo.tensor<[N, 64], f32>)
   index   (%arg6: !map.index<2>)  -> !mo.tensor<[N, 64], f32> {
   %2 = map.load %arg0[%arg6] : !mo.tensor<[N, 64], f32>
   %3 = map.index.transpose<#map.index.transpose.spec<[64, N] -> [N, 64], perm = [1, 0]>> (%arg6, %0) : (!map.index<2>,
!mo.tensor<[2], si64>) -> !map.index<2>
   %4 = map.load %arg1[%3] : !mo.tensor<[64, N], f32>
   %5 = map.call.elementwise @builtin_kernels.elementwise.Add(%arg6, %2, %4) : !map.index<2>, f32, f32 -> f32
   map.store %arg5[%arg6] = %5 : !mo.tensor<[N, 64], f32>
 } {fusionInfo = #mogg.fusion<#mogg.fusion_view<[#mogg.fusion_leaf<"mo.transpose", loc =
loc(".fusion/pipeline/00-input.mlir":3:9), sig = ..., #mogg.fusion_leaf<"mo.add", loc =
loc(".fusion/pipeline/00-input.mlir":4:10), sig = ..., f32>>]>>, ...}
...
```

Keeps track of source information and fusion type

💡 **Takeaway:** Structured kernels (ex: elementwise) are transformed into a more generic form, making it easier to do other kinds of optimizations.

-map-outline-structured-kernel-bodies

```
  %1 = map.iter.opaque
    inputs (%arg0: !mo.tensor<[N, 64], f32>, %arg1: !mo.tensor<[64, N], f32>, %0: !mo.tensor<[2], si64>)
    outputs (%out : !mo.tensor<[N, 64], f32>) {
    map.iter.foreach.body<[N, 64]>
      index   (%arg5: !map.index<2>)
      capturing (%arg6 = %arg0 : !mo.tensor<[N, 64], f32>, %arg7 = %arg1 : !mo.tensor<[64, N], f32>, %arg8 = %0 : !mo.tensor<[2],
si64>, %arg9 = %out : !mo.tensor<[N, 64], f32>) {
      %2 = map.load %arg6[%arg5] : !mo.tensor<[N, 64], f32>
      %3 = map.index.transpose<#map.index.transpose.spec<[64, N] -> [N, 64], perm = [1, 0]>> (%arg5, %arg8) : (!map.index<2>,
!mo.tensor<[2], si64>) -> !map.index<2>
      %4 = map.load %arg7[%3] : !mo.tensor<[64, N], f32>
      %5 = map.call.elementwise @builtin_kernels.elementwise.Add(%arg5, %2, %4) : !map.index<2>, f32, f32 -> f32
      map.store %arg9[%arg5] = %5 : !mo.tensor<[N, 64], f32>
    }
  } {...}
...
```

-map-livm

💡 **Takeaway:** Loop Invariant View Motion reduces index arithmetic in kernels by convert eligible index transformations into stride transformations that exist outside the core loop of a kernel.

```
   %1 = map.iter.opaque
     inputs (...)
     outputs (...) {
...
     map.iter.foreach.body<[N, 64]>
       index   (%arg5: !map.index<2>)
       capturing (...) {
       %2 = map.load %arg6[%arg5] : !mo.tensor<[N, 64], f32>
       %3 = map.index.transpose<#map.index.transpose.spec<[64,
N] -> [N, 64], perm = [1, 0]>> (%arg5, %arg8) : (!map.index<2>,
!mo.tensor<[2], si64>) -> !map.index<2>
       %4 = map.load %arg7[%3] : !mo.tensor<[64, N], f32>
       %5 = map.call.elementwise
@builtin_kernels.elementwise.Add(%arg5, %2, %4) : !map.index<2>,
f32, f32 -> f32
       map.store %arg9[%arg5] = %5 : !mo.tensor<[N, 64], f32>
     }
   } {...}
...
```

```
new_idx = transpose(idx[0], idx[1])
offset  = new_idx[1] * 1 + new_idx[0] * 64
```

```
 %0 = map.iter.opaque
   inputs (...)
   outputs (...) {
...
   %2 =
map.index.transpose.stride<#map.index.transpose.spec<[64, N] ->
[N, 64], perm = [1, 0]>> (%arg1, %1) : (!mo.tensor<[64, N],
f32>, !mo.tensor<[2], si64>) -> !mo.tensor<[N, 64], f32>
   map.iter.foreach.body<[N, 64]>
     index   (%arg4: !map.index<2>)
     capturing (..., %arg9 = %2 : !mo.tensor<[N, 64], f32>) {
     %3 = map.load %arg5[%arg4] : !mo.tensor<[N, 64], f32>
     %4 = map.load %arg9[%arg4] : !mo.tensor<[N, 64], f32>
     %5 = map.call.elementwise
@builtin_kernels.elementwise.Add(%arg4, %3, %4) : !map.index<2>,
f32, f32 -> f32
     map.store %arg8[%arg4] = %5 : !mo.tensor<[N, 64], f32>
   }
 } {...}
...
```

```
offset  = new_idx[1] * 64 + new_idx[0] * 1
```

`-map-to-mogg`

> 💡 **Takeaway:** Kernels are lowered to generic routines that will be transformed into a form that can easily emit Mojo code. This is the “progressive lowering” part.

Getting closer to what we emit in Mojo

```
  %0 = mogg.kernel (%arg0, %arg1 : !mo.tensor<[N, 64], f32>, !mo.tensor<[64, N], f32>) -> !mo.tensor<[N, 64], f32>
    inputs (%arg2: !mogg.pointer_, %arg3: !mogg.pointer_)
    outputs (%arg4: !mogg.pointer_)
    context (%arg6: !mogg.context)
    dims (%arg5: !mogg.dim_<N>) attributes {device = #M.device_ref<"cpu", 0>, fusionInfo = ...} {
    %1 = mogg.shape.create(%arg5) {shape = #mosh<ape[dyn, 64]> : !mosh.ape} : (!mogg.dim_<N>) -> !mogg.shape
    %2 = mogg.tensor.create(%arg2, %1) {device = "cpu", elementType = f32, io = 1 : i32, mut = false, shapeAttr = #mosh<ape[N, 64]> : !mosh.ape} :
(!mogg.pointer_, !mogg.shape) -> !mogg.tensor_
    %3 = mogg.shape.create(%arg5) {shape = #mosh<ape[64, dyn]> : !mosh.ape} : (!mogg.dim_<N>) -> !mogg.shape
    %4 = mogg.tensor.create(%arg3, %3) {device = "cpu", elementType = f32, io = 1 : i32, mut = false, shapeAttr = #mosh<ape[64, N]> : !mosh.ape} :
(!mogg.pointer_, !mogg.shape) -> !mogg.tensor_
...
    %10 = mogg.tensor.create.transpose(%4, %9) {elementType = f32, mut = false, outShape = #mosh<ape[N, 64]> : !mosh.ape, staticPermutations =
array<i64: 1, 0>} : (!mogg.tensor_, !mogg.tensor_) -> !mogg.tensor_
    %11 = mogg.shape.create(%arg5) {shape = #mosh<ape[dyn, 64]> : !mosh.ape} : (!mogg.dim_<N>) -> !mogg.shape
    mogg.call.foreach @builtin_primitives.primitives._foreach shape(%11) context(%arg6) capturing(...) attributes {device = "cpu"} {
    ^bb0(%arg7: !mogg.tensor_, %arg8: !mogg.tensor_, %arg9: !mogg.tensor_, %arg10: !map.index<2>):
      %12 = mogg.tensor.load %arg7[%arg10] : !mogg.tensor_, !map.index<2> -> f32
      %13 = mogg.tensor.load %arg9[%arg10] : !mogg.tensor_, !map.index<2> -> f32
      %14 = mogg.call.elementwise["builtin_kernels.elementwise.Add"] (%12, %13) {moOpName = "mo.add"} : f32, f32 -> f32
      mogg.tensor.store %arg8[%arg10] = %14 : !mogg.tensor_, !map.index<2>, f32
    }
  }
...
```

💡 **Takeaway:** Routines are given a name and outlined into proper functions which makes it possible to deduplicate (faster compilation time).

-mogg-outline-kernels-into-functions

```
   %0 = mogg.kernel (%arg0, %arg1 : !mo.tensor<[N, 64], f32>, !mo.tensor<[64, N], f32>) -> !mo.tensor<[N, 64], f32>
     inputs (%arg2: !mogg.pointer_, %arg3: !mogg.pointer_)
     outputs (%arg4: !mogg.pointer_)
     context (%arg6: !mogg.context)
     dims (%arg5: !mogg.dim_<N>) attributes {device = #M.device_ref<"cpu", 0>, fusionInfo =
#mogg.fusion<#mogg.fusion_view<[#mogg.fusion_leaf<"mo.transpose", loc = #loc, sig = (!mo.tensor<[64, N], f32>, !mo.tensor<[2],
...} {
     %1 = mogg.buffer.constant #M.dense_array<1, 0> : tensor<2xsi64> : !mogg.pointer_
     mogg.call.stub @View_transpose_add_mogg %arg2, %arg3, %arg4, %arg5, %arg6, %1 : !mogg.pointer_, !mogg.pointer_,
!mogg.pointer_, !mogg.dim_<N>, !mogg.context, !mogg.pointer_
   }
   mo.output %0 : !mo.tensor<[N, 64], f32>
 }
 mogg.kernel.stub @View_transpose_add_mogg (%arg0: !mogg.pointer_, %arg1: !mogg.pointer_, %arg2: !mogg.pointer_, %arg3:
!mogg.dim_<N>, %arg4: !mogg.context, %arg5: !mogg.pointer_) fusionInfo = View("mo.transpose":3, "mo.add":4) attributes {abi = 2 :
i32, constantResources = [], device = "cpu", sym_visibility = "private"} {
...
   mogg.call.foreach @builtin_primitives.primitives._foreach shape(%9) context(%arg4) capturing(%1, %5, %8 : !mogg.tensor_,
!mogg.tensor_, !mogg.tensor_) attributes {device = "cpu"} {
   ^bb0(%arg6: !mogg.tensor_, %arg7: !mogg.tensor_, %arg8: !mogg.tensor_, %arg9: !map.index<2>):
...
   }
```

Helps compilation time (function deduplication)

💡 **Takeaway:** Some debugging features are added here (not all shown).

-mogg-wrap-kernels-with-trace-op

```
mogg.kernel.stub @View_transpose_add_mogg (%arg0: !mogg.pointer_, %arg1: !mogg.pointer_, %arg2: !mogg.pointer_, %arg3:
!mogg.dim_<N>, %arg4: !mogg.context, %arg5: !mogg.pointer_) fusionInfo = View("mo.transpose":3, "mo.add":4) attributes {abi = 2 :
i32, constantResources = [], device = "cpu", sym_visibility = "private"} {
...
 mogg.with_trace {
   mogg.call.foreach @builtin_primitives.primitives._foreach shape(%9) context(%arg4) capturing(%1, %5, %8 : !mogg.tensor_,
!mogg.tensor_, !mogg.tensor_) attributes {device = "cpu"} {
   ^bb0(%arg6: !mogg.tensor_, %arg7: !mogg.tensor_, %arg8: !mogg.tensor_, %arg9: !map.index<2>):
     %10 = mogg.tensor.load %arg6[%arg9] : !mogg.tensor_, !map.index<2> -> f32
     %11 = mogg.tensor.load %arg8[%arg9] : !mogg.tensor_, !map.index<2> -> f32
     %12 = mogg.call.elementwise["builtin_kernels.elementwise.Add"] (%10, %11) {moOpName = "mo.add"} : f32, f32 -> f32
     mogg.tensor.store %arg7[%arg9] = %12 : !mogg.tensor_, !map.index<2>, f32
   }
   mogg.yield
 }
 mogg.output
}
```

💡 **Takeaway:** Kernels no longer return new tensors: They receive their pre-allocated outputs (destination passing style). The graph is also now “async”, matching the runtime.

-mogg-to-mgp

- Assertions are inserted
- Allocations are made explicit
- Kernel entry points because generic compiled regions (just functions)

```
 mgp.model @add : mof {...} devices [#M.device_ref<"cpu", 0>] init (...) {
   %0 = grt.chain.create ()
   %1 = mgp.context.create 1
   %2 = mgp.runtime.create %0, %1[0]
   mef.output %2, %1, %arg0, %arg1 : !mef.chain, !mgp.context,
!mgp.opaque<"weights_registry">, !mgp.device_context<<"cpu", 0>>
 } execute (%arg0: !mef.chain, %arg1: !mgp.context, %arg2: !mgp.opaque<"weights_registry">,
%arg3: !mgp.device_context<<"cpu", 0>>, %arg4: !mgp.tensor<?x64xf32>, %arg5:
!mgp.tensor<64x?xf32>) -> (!mef.chain, !mgp.tensor<?x64xf32>) {
   %0 = mgp.tensor.extract.tensor_spec %arg4 : <?x64xf32>
   %1 = mgp.tensor_spec.get_dim[0] %0 : <?x64xf32>
...
   %8 = pop.cmp eq(%3, %7) : <1, si64>
   %9 = pop.cast_to_builtin %8 : !kgen.scalar<bool> to i1
   %10 = mgp.assert %9, "symbolic dimension 'N' for input 1 does not match prior uses of that
dimension"
...
   %21 = mgp.buffer.alloc %arg0, %20, %arg3, align 16 : !mgp.buffer,
!mgp.device_context<<"cpu", 0>>
   %22 = mogg.generic.compiled.region (%arg0, %13, %15, %21, %3, %arg3) : (...)  -> !mef.chain
{
     %24 = mogg.async.unpack(%arg7) : !mgp.buffer
     %25 = mogg.rebind %24 : !mgp.buffer to !mogg.pointer_
     %26 = mogg.async.unpack(%arg8) : !mgp.buffer
     %27 = mogg.rebind %26 : !mgp.buffer to !mogg.pointer_
...
     mogg.call.stub @View_transpose_add_mogg %25, %27, %29, %31, %33, %35 : !mogg.pointer_,
!mogg.pointer_, !mogg.pointer_, !mogg.dim_, !mogg.context, !mogg.pointer_
     %36 = mogg.async.ready : !mef.chain
     mogg.output %36 : !mef.chain
 }
   %23 = mgp.tensor.create %22, %21, %18 : !mgp.tensor<?x64xf32>
   mef.output %arg0, %23 : !mef.chain, !mgp.tensor<?x64xf32>
 }
```

💡 **Takeaway:** Mojo code is emitted and sent to the Mojo compiler. Compiling the Mojo is the slowest part of the compilation pipeline by far (>90%).

-mogg-emit-mojo

```
# FusionInfo:
#   View
#       ├── mo.transpose (.fusion/pipeline/00-input.mlir:3) : (!mo.tensor<[64, N], f32>, !mo.tensor<[2], si64>) -> !mo.tensor<[N, 64], f32>
#       └── mo.add (.fusion/pipeline/00-input.mlir:4) : (!mo.tensor<[N, 64], f32>, !mo.tensor<[N, 64], f32>) -> !mo.tensor<[N, 64], f32>

def View_transpose_add_mogg(arg__0: OpaquePointer[MutAnyOrigin], arg__1: OpaquePointer[MutAnyOrigin], arg__2: OpaquePointer[MutAnyOrigin], arg__3: Int, arg__4:
max.gpu.host.device_context.DeviceContext, arg__5: OpaquePointer[MutAnyOrigin]) raises:
   try:
       var var___0 = layout.Coord(Scalar[DType.int](arg__3), layout.Idx[64])
       var var___1 = layout.row_major(var___0)
       var var___2 = builtin_primitives.primitives.mogg_tensor_init[DType.float32, 2, False, extensibility.IO.Input, 64](arg__0.unsafe_bitcast[NoneType](), var___1)
...
       with max.runtime.tracing.Trace[max.runtime.tracing.TraceLevel.OP, target = StaticString("cpu")]("View(transpose, add)", task_id = ...):
           var var____0 = var___2.unsafe_ptr()
           var var____1 = var___2.shape()
           var var____2 = var___2.strides()
           comptime param____3 = var___2.static_spec
...
           def lambda_____12[width : Int, element_alignment : Int = 1](idx: layout.Coord) {var}:
               var var_____0 = managed_tensor_slice.simd_load_from_tensor_pointer[...](std.utils.coord.coord_to_index_list(idx))
               var var_____1 = managed_tensor_slice.simd_load_from_tensor_pointer[...](std.utils.coord.coord_to_index_list(idx))
               var var_____2 = builtin_kernels.elementwise.Add.elementwise[dtype = DType.float32](var_____0, var_____1)
               managed_tensor_slice.simd_store_into_tensor_pointer[...](var_____2)
           builtin_primitives.primitives._foreach[...](var___0, lambda_____12, arg__4)
   except e__6:
       var var___0 = builtin_primitives.primitives.mogg_format_kernel_error("View_transpose_add_mogg", e__6, "  View\n      ├── mo.transpose
(.fusion/pipeline/00-input.mlir:3) : (!mo.tensor<[64, N], f32>, !mo.tensor<[2], si64>) -> !mo.tensor<[N, 64], f32>\n      └── mo.add (.fusion/pipeline/00-input.mlir:4)
: (!mo.tensor<[N, 64], f32>, !mo.tensor<[N, 64], f32>) -> !mo.tensor<[N, 64], f32>\n", "")
       raise var___0^
```

The Future

Improved optimizations, more features but the same philosophy

Horizontal fusion

```
%y1 = map.iter.foreach<[D0, D1, 128]>
   inputs  (%x  : !mo.tensor<[D0, D1, 128], f32>)
   outputs (%o1 : !mo.tensor<[D0, D1, 128], f32>)
   index   (%idx : !map.index<3>)
   -> !mo.tensor<[D0, D1, 128], f32>
{
 %v = map.load %x[%idx] : !mo.tensor<[D0, D1, 128], f32>
 %e = map.call.elementwise
@builtin_kernels.elementwise.Exp(%idx, %v)
     : !map.index<3>, f32 -> f32
 map.store %o1[%idx] = %e : !mo.tensor<[D0, D1, 128], f32>
}

%y2 = map.iter.foreach<[D0, D1, 128]>
   inputs  (%x  : !mo.tensor<[D0, D1, 128], f32>)
   outputs (%o2 : !mo.tensor<[D0, D1, 128], f32>)
   index   (%idx : !map.index<3>)
   -> !mo.tensor<[D0, D1, 128], f32>
{
 %v = map.load %x[%idx] : !mo.tensor<[D0, D1, 128], f32>
 %s = map.call.elementwise
@builtin_kernels.elementwise.Sqrt(%idx, %v)
     : !map.index<3>, f32 -> f32
 map.store %o2[%idx] = %s : !mo.tensor<[D0, D1, 128], f32>
}
```

```
%y1, %y2 = map.iter.foreach<[D0, D1, 128]>
   inputs  (%x  : !mo.tensor<[D0, D1, 128], f32>)
   outputs (%o1 : !mo.tensor<[D0, D1, 128], f32>,
            %o2 : !mo.tensor<[D0, D1, 128], f32>)
   index   (%idx : !map.index<3>)
   -> !mo.tensor<[D0, D1, 128], f32>, !mo.tensor<[D0, D1,
128], f32>
{
 %v = map.load %x[%idx] : !mo.tensor<[D0, D1, 128], f32>

 %e = map.call.elementwise
@builtin_kernels.elementwise.Exp(%idx, %v)
     : !map.index<3>, f32 -> f32
 %s = map.call.elementwise
@builtin_kernels.elementwise.Sqrt(%idx, %v)
     : !map.index<3>, f32 -> f32

 map.store %o1[%idx] = %e : !mo.tensor<[D0, D1, 128], f32>
 map.store %o2[%idx] = %s : !mo.tensor<[D0, D1, 128], f32>
}
```

💡 **Takeaway:** Kernels that share at least one input but don't have a shared edge can be merged into one kernel. This reduces the total number of loads and can also avoid duplicated computation.

Reductions synthesis (like elementwise)

```
%out = map.iter.rowwise<[D0, D1, 128], axis = 2>
   inputs  (%x   : !mo.tensor<[D0, D1, 128], f32>)
   outputs (%o   : !mo.tensor<[D0, D1], f32>)
   index   (%row : !map.index<2>)
   -> !mo.tensor<[D0, D1], f32>
{
 %sum = map.iter.reduce<[D0, D1, 128], axis = 2>
     row   = %row
     index (%idx : !map.index<3>)
     acc   (%acc : f32) =
#map.join.__init__(@builtin_kernels.reduce.Add)
     -> f32
 {
   %v   = map.load %x[%idx] : !mo.tensor<[D0, D1, 128],
f32>
   %res = map.join @builtin_kernels.reduce.Add(%acc, %v) :
f32
   map.yield %res : f32
 }

 map.store %o[%row] = %sum : !mo.tensor<[D0, D1], f32>
}
```

```
def reduce_sum[
   dtype: DType,
   InputFn: def[width: Int](Coord) -> SIMD[dtype, width],
   OutputFn: def[width: Int](Coord, SIMD[dtype, width]) -> None,
   reduce_dim: Int,
](input_fn: InputFn, output_fn: OutputFn, input_shape: Coord):
   @inline(.always)
   def body[
       params: rowwise.ContextParams
   ](row_coords: Coord, mut ctx: rowwise.Context[params]) {var
input_fn, var output_fn}:
       comptime rank = row_coords.rank

       def load[width: Int](idx: RowCoord[rank]) {var input_fn} ->
SIMD[dtype, width]:
           return input_fn[width](idx.coord)

       var row = rowwise.Row[params, dtype, reduce_dim,
rank](row_coords, ctx, load)

       var acc = row.reduce[ReduceSum[dtype,
params.simd_width]](load).acc

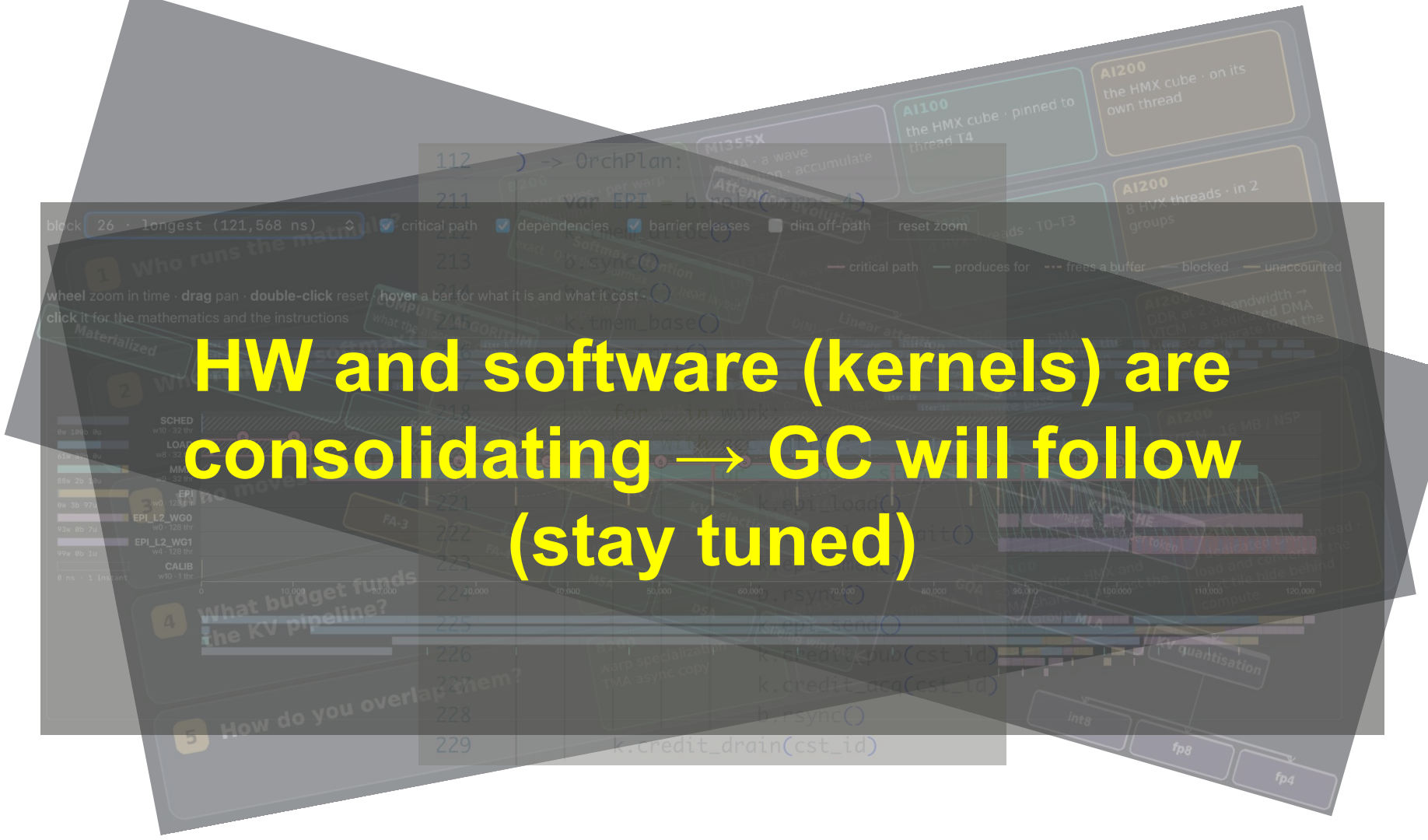
       def write(oc: RowCoord[rank]) {var acc, var output_fn}:
           output_fn[params.emit_tile_width](oc.coord,
acc.slice[params.emit_tile_width]())

       row.emit(write)

   rowwise.launch[axis=reduce_dim, num_phases=1](body, input_shape)
```

💡 **Takeaway:** Like elementwise ops, reductions will be fully synthesized by the GC. This normalizes performance across all reductions and makes it easier to support variants.


AI200
the HMX cube · on its own thread
AI100
the HMX cube · pinned to
AI200
threads · in 2
112) -> OrchPlan:
block 26 · longest (121,568 ns)
critical path
dependencies
barrier releases
dim off-path
reset zoom
critical path
produces for
frees a buffer
blocked
unaccounted
wheel zoom in time · drag pan · double-click reset · hover a bar for what it is and what it cost ·
click it for the mathematics and the instructions
SCHED
LOAD
MMA
EPI
EPI_L2_WG0
EPI_L2_WG1
CALIB
0
10,000
20,000
30,000
40,000
50,000
60,000
70,000
80,000
90,000
100,000
110,000
120,000
229 k.credit_drain(cst_id)
fp8
fp4


HW and software (kernels) are consolidating → GC will follow (stay tuned)

Runtime

Definition

“Runtime”: handles anything we can’t do at compile time:

- Schedule operations with unpredictable latency.
- Allocate memory, manage buffer lifetimes.
- Handle interactions with 3rd party libraries and device drivers.

Runtime Components

- AsyncRT - Asynchronous CPU Runtime
- GraphRT - Graph Executor
- DeviceContext - Accelerator Abstraction Layer

AsyncRT

Asynchronous CPU Runtime

AsyncRT powers compilation and execution

One asynchronous CPU runtime, many users across the stack.

Parallel compilation

Build and optimize code



MLIR / LLVM

thread pool

Mojo

Mojo compiler

Graph compiler

Parallel execution

Run host-side work

Mojo standard library

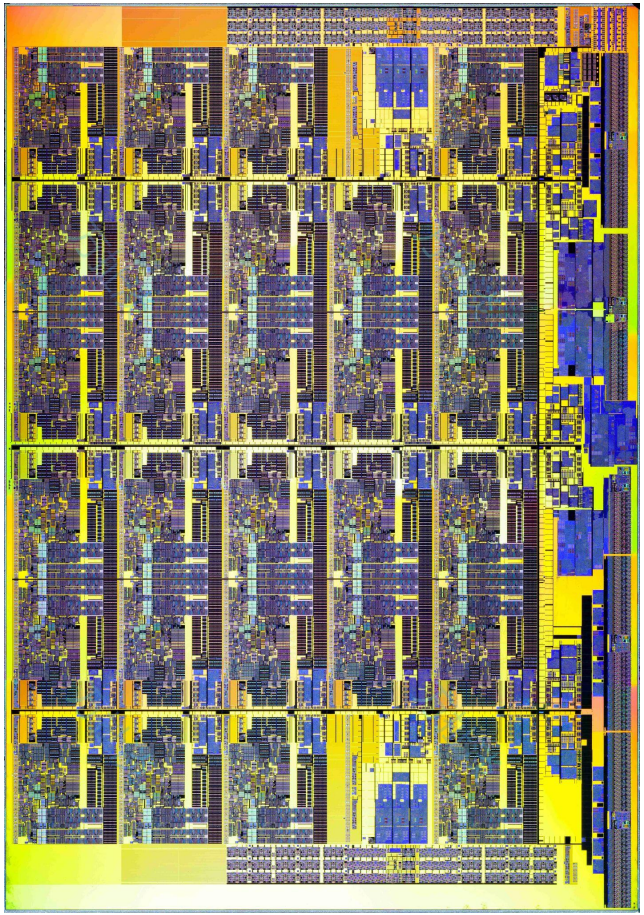
CPU kernels

GraphRT

AsyncRT provides the shared CPU task runtime.

Design Goals of AsyncRT

- Scale efficiently to hundreds of cores.
- Interoperate with 3rd-party runtimes and application code.
- Support multiple host architectures.
- Enable parallelism both within an op and between ops in the graph.
- Avoid performance pitfalls:
  - OS context switches
  - Oversubscription
  - Undersubscription
  - Cache flushes
  - Busy waiting



Intel Copper lake
https://kurnal-insights.com/en/dieshot/?m=Intel&t=cpu&y=era-2018-2020

AsyncRT Work Queue

- One worker thread per core.
- Work items must never block.
- Continuously execute from work queue.

Simple, predictable runtime behavior. Flexible primitives allow the client to customize and optimize.

```
while (!stop) {
  auto lambda = workQueue.atomic_pop_back();
  lambda();
}
```

```
// move-only std::function<void()>
using TaskFunction = llvm::unique_function<void()>;

class WorkQueue {
 /// Enqueue a block of work. Thread-safe.
 virtual void addTask(TaskFunction work) = 0;
};
…

queue.addTask([]() {
  printf(“hello world\n”);
});
```

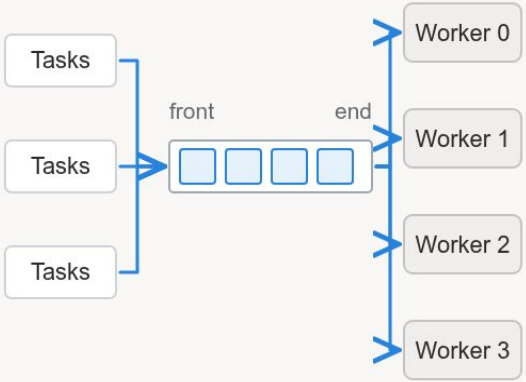
AsyncRT work queues

Tasks enter at the front; workers take work from the end.

Global queue

Many producers → many consumers

MPMC · one shared queue



Enqueue **Dequeue**

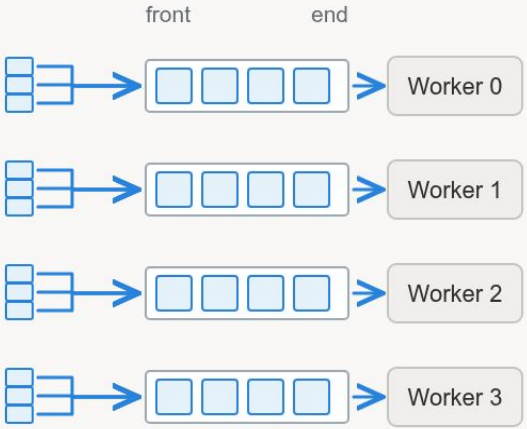
Any producer can submit work.

Any worker can take work.

Affinity queues

Many producers → one consumer

MPSC · one queue per worker



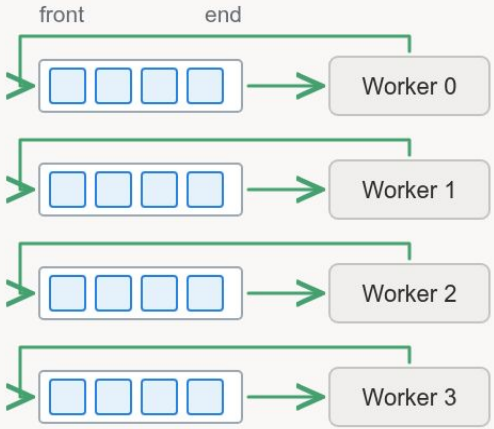
Many tasks target a specific worker.

Only that worker consumes its queue.

Local queues

One producer → one consumer

SPSC · one queue per worker



The same worker enqueues
and consumes its continuation tasks.

Blue: shared / targeted work Green: worker-local continuations

AsyncValue

Refcounted, heap-allocated, type-erased future for a value

- **Refcounted**: Supports multiple consumers, automatic lifetime management.
- **Type-erased**: Can manipulate without knowing the concrete type via **`AnyAsyncValueRef`**
- **Future**: Implements state machine: **`Allocated ➜ Constructed ➜ Ready | Error`**
- **Value**: Any type, even non-copyable.
- **Chain**: Empty AsyncValue, models side-effects.

Source ref: `AsyncValue.h`

Waiters

AsyncRT workers **don’t block** when waiting for an AsyncValue.

Instead, they create a **waiter task** that will run when the value is ready.

- **andThenSync**: execute the continuation inline on the same thread
- **andThenAsync**: schedule a task to run the continuation.

Meanwhile, they can execute other tasks!

AsyncValue: wait without blocking a worker

A future holds a value and the continuations waiting for it.

01 Register a waiter

AsyncValue<T>
NOT READY · payload unavailable

Waiters

- Continuation A
- Continuation B

02 Keep doing useful work

Worker thread

- Task X
- Task Y
- Task Z

No blocked worker. No polling loop.

03 Make the value ready

AsyncValue<T>
READY · payload available

The producer completes the value.

Registered waiters are triggered.

How should the continuation run?

andThenSync

Mark value ready → Run continuation inline

Same thread that makes the pending value ready.

No additional task is queued.

andThenAsync

Mark value ready → Queue task → Worker runs it

The continuation is scheduled as a task.

It may run on the same or a different worker.

If already ready: andThenSync runs inline during registration; andThenAsync schedules a task.

GraphRT

Compiled Graph executor

Compiled Graph Representation

MEF (Modular Executor Format)

- Binary representation of the compiled graph on disk or in memory.
- Equivalent to MGP dialect, but with an optimized representation.

Nodes: Primitives (blobs of compiled code)

Edges: AsyncValue<T> (operand and result values)

Entry points: `init()` and `execute()`

MEF Executor

Dataflow-driven execution, no lexical ordering.

1. Execute a ready primitive:
   - Runs on MEF Executor thread by default.
   - Most primitives return early, creating an async task to do the real work and mark the results ready.
2. Enqueue waiters on result AsyncValues:
   - Task runs only when the result is marked ready.
   - For each user, decrement per-primitive counter tracking “operands not yet ready”.
   - If all operands ready, goto 1.

Source reference: `MEFInvocable.cpp`

`MEFExecutor::processUsedByRecords()`

`MEFExecutor::decrementOperandsNotReadyCounts()`

Strict vs non-strict primitives

Strict primitives must wait for **ALL** inputs to be ready (common)

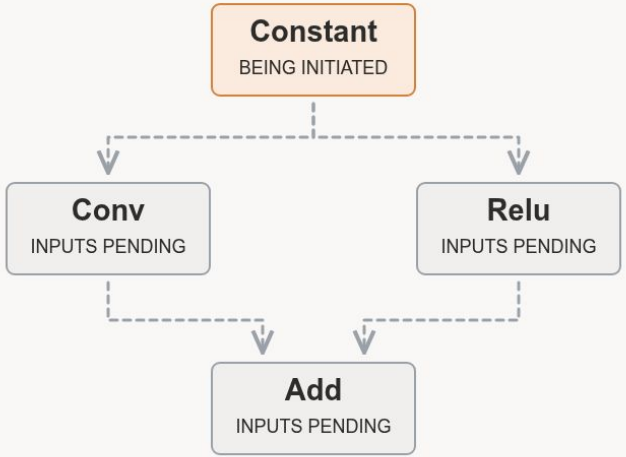
Non-strict primitives wait for **ANY** input to be ready (control flow)

The GC emits code to automatically wrap / unwrap AsyncValues in a compiled region, since they are guaranteed ready. Kernels don’t need to worry about this, and consume the values directly.

Initiate work. Don’t block graph execution

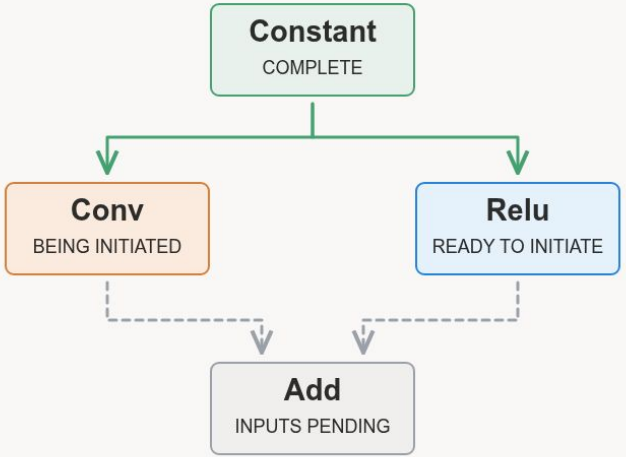
Returning an AsyncValue lets the executor discover other ready work.

01 Completion exposes parallelism



When Constant completes, Conv and Relu can both start.

02 Initiation keeps the executor moving



Initiate Conv, then initiate Relu without waiting for Conv.

Add waits for both result AsyncValues, not for the executor to run nodes in source order.

Launched & complete Being initiated Inputs not ready Ready, not yet initiated

Within the primitive

Primitives contain arbitrary code:

- Mojo kernels.
- Fused kernels synthesized by the graph compiler.
- Calls into 3rd-party kernel library.
- AsyncRT task launches.
- Accelerator kernel launches (via DeviceContext).

Reducing GraphRT overhead

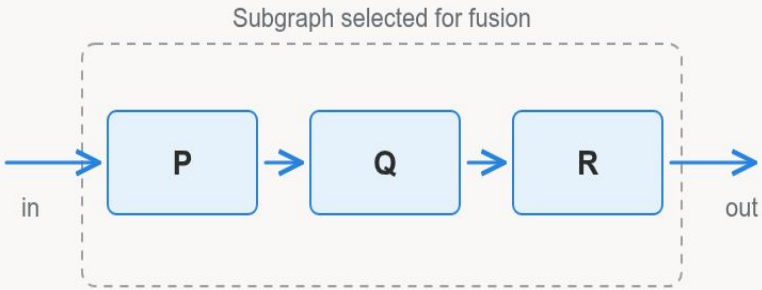
- GraphRT only cares about runtime dependencies! If ops always execute together, we can perform **sequence fusion.**
- Fan-out is best expressed via direct use of AsyncRT, rather than many independent primitives.
- Delegate work to device runtimes where possible:
  - GPU Streams
  - CUDA / HIP Graphs
  - Graph Blobs (coming soon)

Sequence fusion

A sequential subgraph becomes one GraphRT primitive.

Before

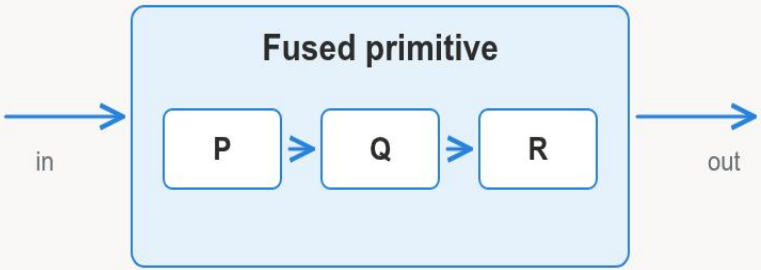
Three primitives, two internal AsyncValue edges



GraphRT schedules P, then Q, then R.

After

One primitive, internal calls execute in sequence



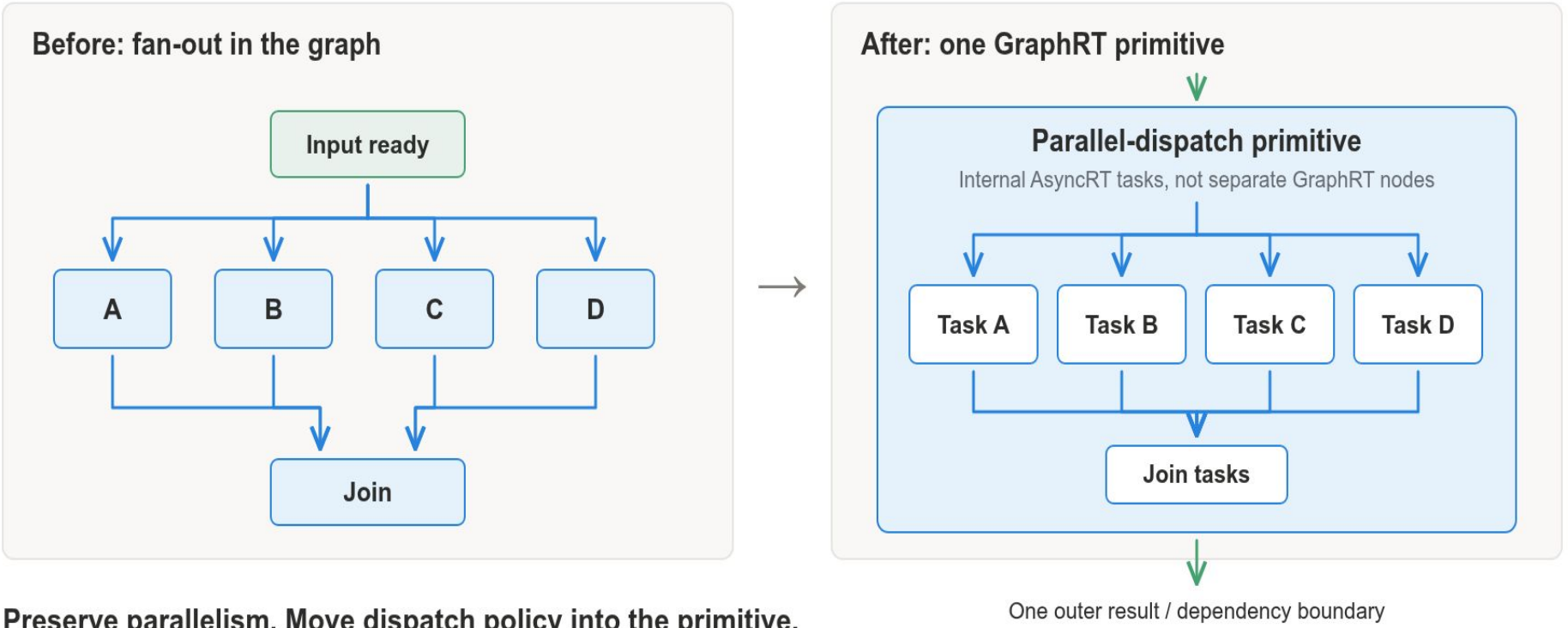
GraphRT schedules the fused region once.

Keep the external dependencies. Remove internal graph scheduling boundaries.

Use when the operations always execute together and internal asynchronous boundaries are unnecessary.

Capture fan-out inside one primitive

GraphRT handles the outer dependency. AsyncRT provides parallel dispatch inside the primitive.

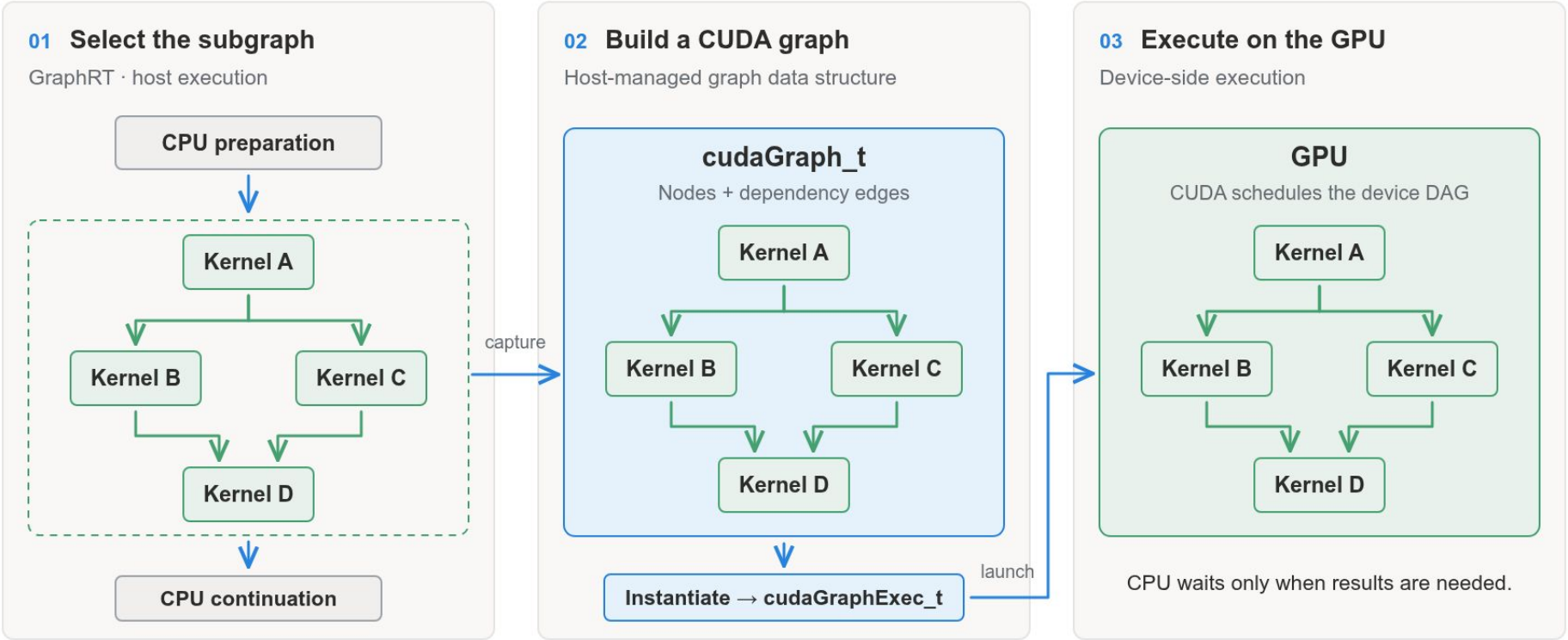


Preserve parallelism. Move dispatch policy into the primitive.

Tasks A–D may execute concurrently. GraphRT does not have to schedule each branch independently.

Offload a device subgraph to a CUDA graph

Capture device operations and their dependencies. Let the GPU execute the subgraph.



GraphRT keeps one launch primitive in place of the captured device subgraph.

CUDA graph handles live on the host. The captured device operations execute on the GPU.

DeviceContext

Abstraction layer for GPU / accelerator driver calls

DeviceContext Implementation

MAX is isolated from the device driver library (e.g. libcuda.so is a runtime dep only)

Single Device API with multiple implementations.

- Generic operations are portable across accelerators.
- Kernels are not tightly coupled to a single device library.
- Underlying access to vendor library is still possible.

DeviceContext API

Key API's:

- DeviceBuffer, host-side refcounted handle to device memory.
- `compile_function()`, prepare pre-compiled Mojo code for launch
- `enqueue_function()`, launch a kernel on the device
- `enqueue_create_buffer()`, create a buffer on the device.
- `enqueue_copy()`, move data between host and device(s)
- `synchronize()`, wait for all pending device work to complete
- Record / wait for event

Note: The current API is heavily influenced by stream semantics on GPU. Work is underway to generalize this for non-GPU accelerators.

Devices in the compiler

- High-level IR (MO, MAP, MOGG) uses abstract device refs (e.g. gpu:0)
  - Tensors have explicit devices, op execution devices are inferred.
- Low-level IR (MGP) adds a DeviceContext argument to each op.
  - Ops receive a DeviceContext for each device they run on.
- At model init, create runtime devices and a context object for each one.

Summary

- AsyncRT
  - Cooperative runtime for efficiently scheduling and executing async tasks.
  - Parallelize the compiler and host-side computations.
- GraphRT
  - Flexible dataflow graph executor, built on AsyncRT.
  - Executes MEF files generated by the graph compiler.
- DeviceContext
  - Hardware abstraction layer for the graph compiler and kernels.
  - Key component for supporting heterogeneous hardware accelerators.